Conditional Copy

A smaller iPRA axiomatization

Research report for Vladimir Reshetnikov

November 12, 2026

Abstract

The current ConciseIPRA collection separately contains introduction at the newest numerical parameter and discharge with a replacement inferred from two matrix rows. We show that these restrictions can coexist. The missing argument is a conditional-copy theorem: for every native formula $H(x)$, the formula $\mathsf{D}_{x,y}\mathsf{Sel}(y, H(t), H(x))$ is uniformly provable without equality transport. Its proof uses a predecessor that computes on successors, followed by one clause specialization. This transfers an arbitrary introduction witness into the newest parameter position without assuming parameter weakening of the restricted calculus.

The resulting presentation has one numerical recursor, equality and one paired formula binder, a fixed reflexivity clause, and three inference families: Use, Close, and Export. Close has no supplied witness. Export has no supplied replacement term, equality proof, or formula template. The ordinary realization is bi-interpretable with full first-order intuitionistic primitive recursive arithmetic; both numerical domains are whole and both composite isomorphisms are the identity. The contribution is a proved compatibility theorem and a smaller interface. A second presentation uses supplied-proof Name at discharge: its complete checker has 1907 normalized bytes, 619 significant tokens, and 686 AST nodes, versus 1925, 630, and 701 for the current parameter-introduction checker. The fully automatic Export presentation has 1844 bytes but more tokens and AST nodes. No absolute minimum is claimed. Executable evidence accompanies the written proof; the new metatheorems are not proof-assistant verified.

The new mathematical step. An arbitrary term can be moved into a fresh witness position through

$$\Delta_t(H) = \mathsf{D}_{x,y}\mathsf{Sel}(y, H(t), H(x)).$$

This packet is a theorem even when $H(t)$ is not. Once $H(t)$ is supplied, the opened packet supplies $H(b)$ at a fresh b. This is conditional copying of a formula, not an assertion that $b = t$.

1 Comparison and exact claim

The baseline is the user-specified directory `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` in ProveIt, pinned to commit `8fa5ade009b5cceba7b68f07b6a834fe77cf10c1`. Its README lists eighty-four delivered packages and an extensive Lean development [1]. The comparison includes the five packages filed on November 12: `deterministic_introduction`, `evaluation_discharge`, `one_body`, `parameter_intro`, and `uniform_recursion`. In particular, neither a three-family rule count nor successor as a recursor abbreviation is new here.

The closest ingredients are `parameter_intro`, which uses the newest parameter for introduction but retains general substitutive discharge, and `first_instance`/`clause_discharge`, which infer a discharge

substitution but retain an arbitrary introduction witness [2, 3]. The new result removes both choices at once.

Interface	`parameter_intro`	Inferred discharge	Proposed K
Introduction witness	Newest / 0	Supplied term	Newest / 0
Discharge substitution	Supplied equality and template	Inferred from rows	Inferred from rows
Equality proof at discharge	Supplied	None	None
Clause specialization term	Supplied	Supplied	Supplied
Inference families	Three	Three	Three

Theorem 1.1 (Main result)**.** *The calculus* K *specified below, and its supplied-proof Name variant* K_N *in Section 9, are complete presentations of an ordinary first-order theory* T_R *that is bi-interpretable with* iPRA*. Both interpretation domains are* $x = x$*, equality is actual equality, and both numerical composites are internally isomorphic to the identity by the graph* $x = y$*. Every formula has an internal round-trip equivalence. Consequently the theories are equiconsistent.*

Here "full first-order" concerns the logical language. Induction in iPRA is quantifier-free induction, not induction for arbitrary formulas. The theory below initially needs induction only for literal equations. The claims about simplicity concern the displayed rule interface and explicit measured comparisons. No lower bound proving this presentation globally simplest is known.

For a direct size improvement over `parameter_intro`, use K_N: it removes the separately supplied source of the discharge equality, fixing that source to the opened witness. Its backward checker is smaller in all three reported coordinates. For fewer proof inputs, use K: both the introduction witness and the discharge replacement are automatic. These are two interfaces to the same theory, not different strength claims.

2 Complete syntax and arithmetic convention

There is one numerical sort. Raw terms are finite expressions

$$t ::= x \mid 0 \mid R(n, a; i, u.h), \qquad S(a) := R(0, a; i, u.0), \qquad \mathbf{1} := S0.$$

Only i, u are bound in h. They are not bound in n or a. The computation equations are

$$R(0, a; i, u.h) = S(a), \tag{1}$$
$$R(Sn, a; i, u.h) = h[n/i, R(n, a; i, u.h)/u]. \tag{2}$$

All substitutions are capture avoiding. The substitution on the right of (2) is simultaneous.

Public terms are full computation normal forms. To define this without a circular rewrite $R(0, a; 0) \to R(0, a; 0)$, temporarily use distinct raw constructors $0, S, R$ and orient (1)–(2). Print the normal constructor Sa as $R(0, a; i, u.0)$. Every rule substitution is followed by normalization N. Thus computation is part of syntax, and the normalizer is part of the trusted implementation and its size.

This binding notation is a staged first-order description language. For each fixed finite body $h(i, u, \vec{p})$ built from earlier descriptions, introduce the fixed first-order function described by $R(n, a; i, u.h)$. Variables range over numbers, never over functions or codes. There is no object-language universal evaluator.

Lemma 2.1 (Normalization)**.** *Every finite well-sorted term has a unique normal form. Normalization preserves scope and satisfies*

$$\mathsf{N}(t[\sigma]) = \mathsf{N}(\mathsf{N}(t)[\mathsf{N} \circ \sigma]).$$

The same statement holds componentwise for formulas.

Proof. Embed the raw syntax into the numerical fragment of the natural-number recursor with initial value Sa and step $(i, u) \mapsto h(i, u)$. The usual reducibility induction is particularly simple here: there is no application of a variable as a function. Computable numerical arguments and a computable finite update give a computable recursor. At zero it reduces to the initial value; at a successor use the predecessor recursion and then the computable update; at a neutral counter use computability of its reducts and proper subterms. Structural induction, allowing computable substitutions for the free variables, gives strong normalization, also under update binders. The two root rules are disjoint and left linear. Parallel reduction commutes with capture-avoiding substitution, so nested reductions and duplicated reducts join. This proves confluence. Uniqueness follows. Substituting into a computation conversion gives a conversion, which proves the displayed coherence law after normalization. No right-hand side introduces a new free variable. □

The operational normalizer evaluates normal children and recursively unfolds a successor counter. Totality is an external statement about finite terms. It does not provide a uniform primitive-recursive bound on normalization of all descriptions. A fuel-bounded checker is included for a primitive-recursive certificate relation.

2.1 Formulas and clauses

Native formulas and two distinguished equations are

$$A ::= (s = t) \mid \mathsf{D}_{x,y} B(x, y), \qquad \top_* := (0 = 0), \quad \bot_* := (0 = \mathbf{1}).$$

The ordinary reading is

$$\mathsf{D}_{x,y} B := \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big). \tag{3}$$

A clause is $[A; y.B(y)]$, read $A \to \forall y B(y)$, with y absent from A. It may have no antecedent: $[\,; y.B(y)]$ means $\forall y B(y)$. An ordinary assumption F is the constant clause $[\,; v.F]$. Contexts are finite reusable lists of clauses. Every context contains the one fixed clause

$$\mathcal{R} = [\,; y.y = y]. \tag{4}$$

An ordered numerical environment records the available free parameters. Write ℓ for its newest parameter, or 0 if that environment is empty. Fresh variables introduced below are absent from the outer environment and context. Formula binders and recursor binders never identify their bound variables with an outer parameter.

3 The three inference families

These rules, the fixed clause (4), and the syntax convention above are the entire presentation. All displayed formulas and instances are normalized.

Use. For a retained clause $[A; y.B(y)]$ and an outer-scope term t,

$$\frac{\Gamma \vdash A}{\Gamma \vdash \mathsf{N}(B(t))}. \tag{Use}$$

There is no premise when the antecedent is empty. In particular, $\mathcal{R}$ proves every $t = t$.

Close. To conclude $\mathsf{D}_{x,y}B$, the witness is automatically ℓ. For a fresh z set

$$\eta = \begin{cases} z, & \mathsf{N}(B(\ell, 0)) \text{ is literally } \top_*, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$\frac{\Gamma, [\,; v.\mathsf{N}(B(\ell, \eta))] \vdash \mathsf{N}(B(\ell, Sz))}{\Gamma \vdash \mathsf{D}_{x,y}B}. \tag{Close}$$

The selected witness is the newest *outer* parameter, chosen before z is adjoined. The new z is available only in the premise. The instruction supplies neither a witness nor a mode. The exact test $\top_*$ is narrower than "any reflexive equation" and suffices for completeness.

Export. Prove a major $\mathsf{D}_{x,y}B$ and open it at fresh b. Compute

$$B_0 = \mathsf{N}(B(b, 0)), \qquad B_1 = \mathsf{N}(B(b, \mathbf{1})), \qquad r_B = \begin{cases} t, & B_0 \equiv \top_* \text{ and } B_1 \equiv (b = t), \\ b, & \text{otherwise.} \end{cases}$$

Here $\equiv$ means literal identity of normal syntax, including the orientation of the equation in the first case. Add the clause $\mathcal{C}_B(b) = [B_0; y.\mathsf{N}(B(b, y))]$ and infer

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B \qquad \Gamma, \mathcal{C}_B(b) \vdash F}{\Gamma \vdash \mathsf{N}(F[r_B/b])}, \qquad b \notin \mathrm{FV}(\mathsf{N}(F[r_B/b])). \tag{Export}$$

The branch formula F may mention b. The inferred term r_B may also mention b; substitution is one simultaneous pass and does not recurse into a newly inserted copy. The normalized output must lie in the outer scope. No equality proof, replacement term, or separate formula template is supplied to this rule.

Lemma 3.1 (Soundness)**.** *Every inference of* K *is valid in intuitionistic equality logic with the recursor equations, numerical coverage, separation of* 0 *and* 1*, and contextual induction for literal equations.*

Proof. Use is implication and universal elimination. In Close's case mode, numerical coverage combines the assumed zero row and the supplied successor branch to give the required conditional universal. If the zero row is $\top_*$, the matrix must be a literal equation, because normalization does not erase formula binders. Its zero row is provable and equation induction applies to the given progressive premise. Thus no quantified induction is used. The automatically chosen ℓ is a legal existential witness in both cases.

For Export, open the existential in (3). In the selected case reflexivity proves B_0, and specialization at 1 gives $b = r_B$. Equality substitution changes F into $F[r_B/b]$. In the default case this is simply substitution by b itself. Existential elimination is then legal because the normalized result does not contain b. Normalization is internally justified by the computation equations and fixed-update extensionality (Section 8). □

4 Equality-free preparations

The following terms have useful literal normal-form equations:

$$P(n) = R(n, 0; i, u.i), \qquad P(0) \equiv 1, \qquad P(Sn) \equiv n,$$
$$C(n, a, b) = P(R(n, a; i, u.Sb)), \qquad C(0, a, b) \equiv a, \qquad C(Sn, a, b) \equiv b,$$
$$J(n, a; i, u.h) = P(R(n, a; j, v.S(h[j/i, P(v)/u]))).$$

The last macro, with simultaneous substitution and fresh j, v, satisfies

$$J(0, a; h) \equiv a, \qquad J(Sn, a; h) \equiv h[n/i, J(n, a; h)/u].$$

Notice $P(0) = 1$, not 0. A conventional predecessor is $p(n) = C(n, 0, P(n))$. The scalar iterator is $I(n, a; u.h) = J(n, a; i, u.h)$ with unused counter coordinate i.

Ordinary elimination. If F is independent of an opened witness b, Export's substitution does nothing to it. Thus ordinary D elimination is available, without knowing which row pattern Export selects. Write

$$\mathsf{q}_y A(y) := \mathsf{D}_{d,y} A(y)$$

with unused d. Opening it and using the resulting clause gives $A(t)$ from $\mathsf{q}_y A$ and $A(0)$. The witness selected by Close is irrelevant for this unused-witness binder.

Numerical cases. Given persistent constructions of $A(0)$ and $A(Sz)$ in their respective scopes, Close introduces $\mathsf{q}_y A(y)$, then Use gives $A(t)$. If its zero row is $\top_*$, supply reflexivity in place of the old zero-row assumption and ignore the progressive assumption. Otherwise the rule is already in case mode. The same observation permits case introduction of q from a branch that assumes $A(0)$.

Gates and truth towers. Every native formula is a finite chain of D binders ending in one equation. Gate a formula A by replacing each side s of its final equation by $C(v, 0, s)$, lifting v under intervening binders. Call the result $\mathsf{G}_v A$. If A has d binders, then

$$\mathsf{G}_0 A \equiv T_d, \qquad \mathsf{G}_{Sz} A \equiv A, \qquad T_0 = \top_*, \quad T_{d+1} = \mathsf{D}_{x,y} T_d.$$

Every T_d has a uniform proof in any environment: use reflexivity at depth zero and Close above it. None of its witnesses occurs in its body.

Selectors. For formulas A, B with the same number of D binders, freshen and align those binders and replace corresponding equation sides s, t by $C(v, s, t)$. This defines $\mathsf{Sel}(v, A, B)$ with literal endpoints

$$\mathsf{Sel}(0, A, B) \equiv A, \qquad \mathsf{Sel}(Sz, A, B) \equiv B.$$

Substitution in numerical terms does not change the number of formula binders. Consequently $H(s)$ and $H(x)$ always have the same depth; the construction at the heart of this paper needs no padding. For general logical encodings, padding is available: $L(A) = \mathsf{q}_v \mathsf{G}_v A$ is interderivable with A using its truth zero row and specialization at 1. Iterating L equalizes depths.

All constructions in this section are explicit builders in their actual contexts. They do not assume that an arbitrary completed K proof can be weakened by appending a newer numerical parameter.

5 The conditional-copy theorem

Theorem 5.1 (Conditional copy)**.** *For every native one-place formula $H(x)$ and every well-scoped term t, the formula*

$$\Delta_t(H) := \mathsf{D}_{x,y}\mathsf{Sel}(y, H(t), H(x))$$

has a uniform K *proof in any ambient environment and context. The construction uses no equality transport and no nontrivial equality premise.*

Proof. Construct the unused-witness closure

$$\mathsf{q}_v\mathsf{G}_v\Delta_{P(v)}(H). \tag{5}$$

Its zero instance is a truth tower. In its successor branch, $v = Sz$ with fresh z, normalization makes the required conclusion exactly $\Delta_z(H)$. The parameter z is now newest. Close of that inner formula therefore selects $x = z$. Its zero row and successor row are both literally $H(z)$:

$$\mathsf{Sel}(0, H(z), H(z)) \equiv H(z), \qquad \mathsf{Sel}(Sw, H(z), H(z)) \equiv H(z).$$

If $H(z) \not\equiv \top_*$, the inner rule is in case mode and its local assumption proves its conclusion by Use. If $H(z) \equiv \top_*$, the rule selects progress and reflexivity proves its successor row. The outer rule is in case mode, since its zero row still has at least one D binder. Its local base assumption is unused.

Open (5) by ordinary elimination, supply the truth tower zero row, and use its clause at $S(t)$. The gate disappears and $P(S(t)) \equiv t$, so the result is exactly $\Delta_t(H)$. It is independent of the closure's unused fresh witness, which is discharged. Every step can be regenerated in any ambient scope. □

This theorem asserts $\exists x(H(t) \to H(x))$ in a deliberately useful native form. It does not prove $H(t)$ and does not identify x with t. Opening the theorem produces a fresh parameter at which the available instance of H can be copied. That is precisely what the newest-parameter restriction needs.

6 Removing arbitrary introduction witnesses

Let L have exactly the syntax, Use, and Export of K, but let Close supply an arbitrary outer term t in place of ℓ. Its mode test is the same: progress exactly when $\mathsf{N}(B(t, 0)) \equiv \top_*$.

Theorem 6.1 (Witness removal)**.** *For every numerical environment, clause context Γ, and native formula A, the calculi* K *and* L *derive the same sequent $\Gamma \vdash A$.*

Proof. Every K Close is an instance of liberal Close, so one direction is immediate. For the other direction, translate a liberal derivation by induction on its finite proof tree. The stronger induction statement allows an injective renaming θ of source numerical parameters into an arbitrary target environment and a map of source clause positions to target clause positions. The clause at every mapped target position must be literally the normalized θ-renaming of its source clause. Additional target parameters and clauses are allowed. This is a membership invariant, not an assumption of arbitrary weakening of already completed target derivations.

Use follows the clause-position map and translates its antecedent proof. Export translates its major, maps the old fresh parameter to the actual new target parameter, and extends the clause-position map by the opened clause. Its two row tests are preserved and reflected by injective renaming and coherent normalization. The only substantive case is a source Close at t, with matrix $B(x, y)$ and branch

$$\Gamma, B(t, \eta) \vdash B(t, Sz).$$

Put $H(x) = \mathsf{q}_y B(x, y)$. The translated source branch proves $H(t)$: the new q witness is unused, and its zero-row test is exactly the source test at t. In particular its assumption and successor goal agree with the original branch after renaming.

Now prove $\Delta_t(H)$ by Theorem 5.1 and open it at fresh b. Its clause is

$$[H(t); v.\mathsf{Sel}(v, H(t), H(b))].$$

Specialization at 1 will give $H(b)$ whenever $H(t)$ is supplied. Immediately introduce the desired $\mathsf{D}_{x,y}B$. The newest outer parameter is b, so the restricted Close chooses exactly b.

In its successor branch a further fresh z is present. Reconstruct $H(t)$ there by the induction hypothesis on the original source branch, then obtain $H(b)$ from the retained packet clause. Open $H(b)$; its clause is $[B(b, 0); y.B(b, y)]$. If the enclosing Close selected cases, its local assumption supplies $B(b, 0)$. If it selected progress, that zero row is literally $\top_*$ and reflexivity supplies it. Use at Sz therefore gives $B(b, Sz)$. Discharge the unused witness of $H(b)$. Close now gives $\mathsf{D}_{x,y}B$, independent of b, and ordinary elimination discharges the packet witness.

Every recursive translation call just described is on the original source Close's strictly smaller branch. The extra packet and truth proofs are explicit finite constructions. Thus the induction is well founded, even though a smaller source proof may be regenerated several times. At each recursive call the source branch's fresh parameter is mapped to the parameter just introduced by the reifying q-Close, and its local assumption is literally that q-Close's local clause. The mode tests agree there because they use the same instantiated zero row. This verifies the clause-membership invariant and completes the well-founded translation. The delivered executable translator implements these numerical and clause-position maps explicitly. □

Remark 6.2 (Why the mode mismatch is harmless). The test at the original term t need not agree with the test at b. The proof transfers the whole conditional closure H, not a progressive premise. At b a progressive mode has a reflexive zero row, so this closure still yields the required successor conclusion. No equality reasoning repairs a failed mode test.

The argument also applies to supplied-proof Name in place of Export. Its stronger consequence here is that introduction and discharge can both be automatic. A naive composition had appeared circular: obtaining $h(b) = u$ from $h(t) = u$ under $b = t$ would already need equality transport. Conditional copy avoids that task altogether.

7 Completeness of the liberal comparison calculus

This section makes explicit the inherited comparison and the small adaptations needed for L. The scalar native kernel, its full-normal presentation, its deterministic Export rule, and the ordinary arithmetic bridge are proved in the pinned repository [1, 6, 3]. Counter-aware R syntax and the exact-zero introduction selector have written treatments in the research packages [4, 2, 5]. The following derivations explain why these ingredients apply together, rather than treating a combination of existing checker programs as a proof.

7.1 Structural facts in the liberal calculus

Clause weakening and injective parameter renaming are direct inductions on liberal derivations. The supplied witness remains available after renaming. Injectivity preserves and reflects both literal row tests, and normalization commutes with renaming. Thus adding fresh parameters is admissible here. Cutting an ordinary assumption $[\,; v.A]$ is then another direct induction: replace a Use of that clause

by its supplied proof and weaken that proof into local scopes using the already established structural facts. These proofs use no equality transport and no arbitrary numerical substitution into proof trees. They apply to L at this stage, not automatically to K.

7.2 Equality from an inferred discharge

Given equations $s = t$ and $h(s) = u$, use the matrix

$$E(x, y): \quad C(y, 0, C(P(y), x, h(x))) = C(y, 0, C(P(y), t, u)). \tag{6}$$

Its normal rows are

$$E(x, 0) \equiv \top_*, \qquad E(x, 1) \equiv x = t, \qquad E(x, S(Sv)) \equiv h(x) = u.$$

In L, introduce $\mathsf{D}_{x,y}E$ at the supplied witness s. The mode is progress. Its successor goal is proved by numerical cases on the predecessor counter, using the two supplied equations; the local progress assumption is unused. Open the packet at b. Its first two rows cause Export to select t. Use at 2, with reflexive zero-row premise, yields $h(b) = u$. Export concludes $h(t) = u$.

This is ordinary one-term transport. Reflexivity and this rule yield symmetry, transitivity, and congruence. Equation-template transport is two such replacements. Full formula transport follows by induction on formula depth: for a D formula open its old witness, introduce at that witness, transport the new zero-row assumption backwards through the smaller matrix, use the old closure at the successor, and transport the result forwards. If the new zero row is $\top_*$, supply reflexivity instead. The final formula is independent of the opened witness, so ordinary elimination discharges it.

The liberal structural facts justify moving supplied premises into the local scopes of this construction. When the result is later translated to K, Theorem 6.1 regenerates its branches in the actual target scopes. Liberal parameter weakening is not being silently assigned to the restricted calculus.

7.3 Cases, logic, and equation induction

Liberal Close simulates arbitrary case introduction: if a case introduction happens to select progress because its base is $\top_*$, replace the old base assumption by reflexivity and ignore the new progress assumption. Ordinary D elimination and closure Use were already derived in Section 4.

For equation induction on $E(y) : s(y) = t(y)$, retain its given base $E(0)$ and put

$$K(y): \quad C(y, 0, s(y)) = C(y, 0, t(y)).$$

Its zero row is literally $\top_*$. The following decoder equations are proved by numerical cases, without induction:

$$C(y, s(0), C(y, 0, s(y))) = s(y),$$
$$C(y, s(0), C(y, 0, t(y))) = t(y).$$

The first has reflexive zero and successor branches. The second uses $E(0)$ in the zero branch and reflexivity at successors. From $K(z)$, congruence with $v \mapsto C(z, s(0), v)$ and the two decoders give $E(z)$. Cut this into the supplied step proof $E(z) \vdash E(Sz)$; the conclusion is literally $K(Sz)$. Primitive progress therefore introduces $\mathsf{q}_y K(y)$. Its reflexive zero row lets Use give $K(a)$ at any target a, and the decoders recover $E(a)$. This proves contextual literal-equation induction without substituting a compound term into an arbitrary Export derivation. If a conditional closure $\mathsf{q}_y E(y)$ is wanted,

introduce it by cases: retain $E(0)$ as its local base and use the just-derived induction rule for the successor goal.

For completeness, the ordinary logical vocabulary has explicit native macros. Equalize binder depths by L before each binary selector. The selector's literal endpoints are these padded formulas; their interderivability with the original operands supplies the connective interfaces stated below:

$$\begin{aligned}(A \to B)^* &= \mathsf{q}_y\mathsf{Sel}(y, A^*, B^*),\\ (A \wedge B)^* &= \mathsf{q}_y\mathsf{G}_y\mathsf{Sel}(P(y), A^*, B^*),\\ (\exists x A)^* &= \mathsf{D}_{x,y}\mathsf{G}_y A^*,\\ (A \vee B)^* &= (\exists v\, \mathsf{Sel}(v, A^*, B^*))^*,\\ (\forall x A)^* &= \mathsf{q}_y\mathsf{G}_y A^*(P(y)), \qquad \bot^* = \bot_*.\end{aligned}$$

In the disjunction line the star denotes expansion of the already defined existential macro, not a second translation of A^*, B^*. Conjunction's rows are truth, A^*, and then B^*, at $0, 1$, and all $S(Sv)$ respectively. Two numerical case splits introduce it; Use at $1, 2$ eliminates it. Implication's zero row is its antecedent and its positive rows its consequent. The existential gate has a truth base. Numerical coverage gives the disjunction elimination rule. Transport along $0 = 1$ changes the provable gate $\mathsf{G}_0 F$ into $\mathsf{G}_1 F \equiv F$, proving explosion for every native F. These give the standard intuitionistic introduction and elimination rules under arbitrary retained assumptions. There is no excluded-middle rule for quantified formulas.

The ordinary reading of each macro is equivalent to the intended connective: split numerical coverage for its selector, specialize at the indicated small numerals for the reverse implications, and use truth of the gate's zero row. The universal macro has a truth zero row and successor row $A^*(z)$ because $P(Sz) \equiv z$; use it at $S(t)$ to obtain $A^*(t)$. Its introduction needs only a fresh renaming of a proof of $A^*(z)$, not substitution into an arbitrary proof tree. Induction on formula construction proves these equivalences in both directions. Expanding the ordinary reading of D back through the macros gives the original D by its derived introduction and elimination rules.

7.4 The exact inherited dependency

The comparison above can be organized as a relative translation of the repository's substitution-stable native diagonal kernel. Computations become reflexivity after normalization; hypotheses use their builders; equality uses (6); ordinary closure and witness elimination use the derived rules; diagonal progress has literal zero row $0 = 0$; case introduction uses the reflexive-base repair just stated. This supplies the full native consequence relation, not only closed equations. Extending the numerical vocabulary by fixed R descriptions is legitimate: each is an ordinary primitive-recursive definition, and the native logical derivations treat numerical terms parametrically. Section 8 supplies both the direct compiler and update extensionality. Thus the comparison includes all R formulas, not only the image of scalar iterator terms.

Proposition 7.1. *The liberal calculus presents T_R, the ordinary intuitionistic first-order equality theory over fixed R descriptions with (1)–(2), numerical coverage, $0 = 1 \to \bot$, and contextual literal-equation induction.*

The soundness proof is Lemma 3.1 with a supplied witness. The converse is the explicit native comparison above. Combined with Theorem 6.1, it establishes the same presentation theorem for K. The new restriction theorem is a written proof; the upstream Lean theorems certify the inherited scalar comparison, not this new recursor calculus or its Python certificate format.

8 Whole-domain bi-interpretation with iPRA

8.1 Compilers and internal computation

Interpret each fixed R description in iPRA by primitive recursion with initial value Sa and update $h(i,u)$, after compiling its proper subdescriptions. Conversely, compile initial functions and composition directly and a fixed primitive-recursive definition by

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$
$$f^*(n,\vec{p}) = J(n, g^*(\vec{p}); i, u.h^*(i,u,\vec{p})).$$

No pairing is necessary because the update sees the old counter. The equations for p and C are immediate. Applying P proves successor injectivity. If $Sx = 0$, applying $C(-,0,1)$ gives $1 = 0$, and separation yields falsity.

Reductions inside an update body must also be internally valid. If $h(i,u) = k(i,u)$ holds uniformly, equation induction on $R(n,a;h) = R(n,a;k)$ proves update extensionality: their bases are both Sa; their successor values agree by the induction equation and the pointwise identity. Each finite normalization conversion therefore has an internal equality proof, including conversions under binders. This justifies treating normal forms as canonical syntax without adding a uniform evaluation axiom.

8.2 Recovering quantifier-free induction

Here are the arithmetic ingredients of the inherited equality-reflection argument, included to make the strength claim checkable. Define addition and monus by J:

$$x + 0 = x, \quad x + Sy = S(x+y), \qquad d(x,0) = x, \quad d(x,Sy) = p(d(x,y)).$$

Put $\mathrm{sg}(u) = C(u,0,1)$ and $K(u) = C(u,1,p(u))$. Coverage and constructor separation give

$$\mathrm{sg}(u) \in \{0,1\}, \quad \mathrm{sg}(u) = 0 \leftrightarrow u = 0, \quad K(u) = 0 \leftrightarrow u = 1.$$

Here the finite-set notation abbreviates an ordinary disjunction. Four literal-equation inductions establish

$$0 + y = y, \quad Sx + y = S(x+y), \quad d(Sx,Sy) = d(x,y), \quad d(Sa,a) = 1.$$

For example, applying p to the induction hypothesis proves the monus cancellation step; the diagonal identity then follows by cancellation.

The useful residual invariant is the single equation

$$C(\mathrm{sg}(d(x,y)), x, y + d(x,y)) = x. \tag{7}$$

Induct on y, holding x fixed. At zero, $0 + x = x$ makes both selector alternatives equal. For the step set $r = d(x,y)$ and $r' = p(r) = d(x,Sy)$. If $r' = 0$, the target selector returns x. Otherwise coverage gives $r = Sr'$ (the zero alternative contradicts $r' \neq 0$). The induction equation gives $y + r = x$, and $Sy + r' = y + Sr' = x$ proves the successor invariant. All case splits are coverage; the induction formula remains (7).

It follows that $d(Sa,b) = 1 \leftrightarrow a = b$: the forward direction uses (7) to get $b + 1 = Sa$, then injectivity; the reverse direction is the diagonal identity. Therefore

$$e(a,b) = \mathrm{sg}(K(d(Sa,b)))$$

has values $0, 1$ and satisfies $e(a, b) = 0 \leftrightarrow a = b$ throughout the entire numerical domain. This is actual equality reflection, not agreement only at standard numerals.

For a quantifier-free formula A, define its Boolean characteristic externally by finite recursion on its syntax:

$$b_{s=t} = e(s, t), \qquad b_{\bot} = 1,$$
$$b_{A\wedge B} = C(b_A, b_B, 1), \qquad b_{A\vee B} = C(b_A, 0, b_B), \qquad b_{A\to B} = C(b_A, b_B, 0).$$

Induction on this finite syntax proves internally that b_A is Boolean and $A \leftrightarrow b_A = 0$. For implication, split the already proved Boolean range of b_A: if it is zero, A holds and the implication is equivalent to B; if it is one, $\neg A$ holds and the implication is true. This uses no excluded middle for arbitrary formulas. Literal-equation induction on $b_A(n) = 0$ now gives every contextual quantifier-free induction instance. Contexts may contain quantified formulas.

8.3 Both round trips, internally

For two fixed recursive descriptions with equal initial values and provably identical updates, equation induction on equality of their values proves uniqueness at every internal counter. Apply this lemma while inducting externally on a finite description: each double translation has the same base and step as its original. This proves both numerical round-trip identities in the respective theories, with all passive parameters arbitrary.

Both domain formulas are $x = x$ and both interpretations preserve actual equality. Hence the composite isomorphism in either direction is the identity graph $x = y$. The connective equivalences in Section 7, followed by induction on each full formula, give both formula round trips. This proves bi-interpretability, not merely mutual conservation of closed equations or agreement in the standard model. Falsity translates to $0 = 1$, whose ordinary realization implies $\bot$ by the explicit separation axiom. Finite contradiction proofs translate in both directions, establishing equiconsistency and Theorem 1.1.

9 A smaller supplied-proof Name presentation

Let K_N keep precisely the syntax, fixed reflexivity clause, Use rule, and newest-only Close rule of K. Replace Export by

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B \qquad \Gamma, \mathcal{C}_B(b) \vdash b = t \qquad \Gamma, \mathcal{C}_B(b) \vdash F}{\Gamma \vdash \mathsf{N}(F[t/b])}, \quad b \notin \mathrm{FV}(\mathsf{N}(F[t/b])). \tag{Name}$$

The replacement t is any well-scoped local numerical term; it may contain b. Substitution inserts it once. The source of the equality is fixed to b, whereas general substitutive discharge supplies both sides of an equality and a formula template.

Proposition 9.1. *The open consequences of K_N, K, and T_R coincide under their displayed readings. In particular, K_N is bi-interpretable with* iPRA *with the same whole domains and identity graphs.*

Proof. Name is sound in T_R: equality gives the substituted branch formula, and the outer-scope condition permits existential elimination. Every Export instance has a Name replacement in the same scope. In the default case use reflexivity $b = b$. In the selected case the opened clause has base $0 = 0$ and first row $b = t$; reflexivity and Use at 1 give the required equality. This translation adds no numerical scope, so it does not change any Close witness or mode. Induction on the proof gives $\mathsf{K} \subseteq \mathsf{K}_N$. Completeness of K and soundness of K_N give the reverse consequence inclusion and the stated interpretation. □

9.1 Why the compact backward certificate is exact

The delivered backward checker supplies a template $T(v)$ and term t, checks the two branch conclusions $b = t$ and $F = \mathsf{N}(T[b/v])$, and requires the outer goal to be $A = \mathsf{N}(T[t/v])$. It does not separately forbid explicit occurrences of b in T. The following lemma explains why that omission preserves exactly Name, even when t contains b.

Lemma 9.2 (Template representation)**.** *For a well-scoped template $T(v)$ and local term t, put $F = \mathsf{N}(T[b/v])$ and $A = \mathsf{N}(T[t/v])$. If A is b-free, then*

$$\mathsf{N}(F[t/b]) = A$$

as literal normal syntax.

Proof. An arbitrary capture-free renaming of numerical parameters preserves a full normal form. Injectivity is unnecessary: recursor reduction tests only zero and successor constructors, and identification of parameters creates neither. Equality formulas have no reduction turning equal sides into $0 = 0$.

Choose a fresh c absent from T, t and set $U = \mathsf{N}(T[v := t, b := c])$, with simultaneous substitution. Identifying c with b therefore gives the already normal formula

$$U[c \mapsto b] = \mathsf{N}(U[c \mapsto b]) = \mathsf{N}(T[t/v]) = A.$$

Since A is b-free, U contains neither b nor c; hence $U = A$. Substitute t for c and use normalization coherence to get $\mathsf{N}(T[v := t, b := t]) = A$. Finally

$$\mathsf{N}(F[t/b]) = \mathsf{N}((T[b/v])[t/b]) = \mathsf{N}(T[v := t, b := t]) = A.$$

No substitution is performed inside a newly inserted copy of t. □

The checker validates templates through their normalized endpoint formulas, as in the existing `parameter_intro` protocol [2]. Here is the needed completion argument. The traversal checks every proper child at its required sort before an erasing reduction. A successful formula endpoint also forces the original root to be an equation or D: a numerical root stays numerical, and a list of any other length cannot reduce to a formula. Thus only excess numerical parameters and reducible numerical terms can remain in a raw accepted template.

Interpret excess negative levels and positive indices above the placeholder's binding depth as extra free parameters. Replace all of them by 0, fixing the placeholder and permitted local parameters. The checked arguments b, t and both checked endpoint formulas contain none of these extras. Normalization coherence therefore preserves both endpoints literally. Normalizing the completed template gives a well-scoped template to which Lemma 9.2 applies. The equality premise validates t, and each endpoint is independently validated in its proper scope; these checks are essential. Conversely, abstracting all free occurrences of b in F into v represents every Name instance. Thus the backward representation is exact, not merely sound on the delivered examples.

This profile retains the replacement, equality proof, and template at discharge. Its advantage is a smaller complete implementation relative to the current backward parameter-introduction checker. Conditional copy is what permits the restricted introduction and restricted equality source to coexist without losing arithmetic or first-order logic.

10 Checker contract and measured comparison

The complete checker reads one JSON value $[goal, proof]$. Terms are integers or ternary lists: 0 is zero; negative integers are rigid free parameters; positive integers are bound de Bruijn indices; $[n, a, h]$ is $R(n, a; i, u.h)$, with u at index 1 and i at index 2. The canonical successor is $[0, a, 0]$. An equation is $[s, t]$ and a paired formula is $[B]$. Both R and D bind two variables in their final child.

The selected compact forward interface has three packet shapes, detailed in `FORMAT.txt`. Use chooses a retained clause and a term. Close contains its matrix and branch, with no witness choice. Export contains its major and branch. The checker infers and validates every conclusion. The supplied goal is compared type-exactly with that validated conclusion, so a second full goal traversal is unnecessary. In particular Python's equality between `True` and `1` is not accepted as syntax. Validation traverses children before an erasing computation; malformed discarded bodies are rejected. Successful conclusion validation enforces Export's outer-scope condition.

Measurements use CPython 3.12.14, remove actual docstrings and comments, apply `ast.unparse`, and add one final LF. Significant tokens omit layout; AST counts include operator and context nodes. Reproduction is `python3 validation/measure.py`.

Complete module	Bytes	Tokens	AST nodes
Automatic Close/Export, K	**1844**	646	748
Backward Name, K_N	**1907**	**619**	**686**
Matched supplied-witness Close	1863	651	755
Matched explicit equality Name	1835	634	742
Selected, with explicit fuel	1995	685	796
Current `parameter_intro`	1925	630	701
Current `uniform_recursion`	1950	662	741
Existing compact scalar control	508	267	302

The backward Name profile saves 18 bytes, 11 tokens, and 15 AST nodes against `parameter_intro`. This is a coordinatewise improvement under the stated complete-source metric. It fixes the equality source to the opened witness and narrows the introduction selector to literal $0 = 0$; it does not merely rename program variables.

For the automatic Export profile, deleting the introduction witness saves 19 bytes, five tokens, and seven AST nodes against the otherwise matched control. This implementation is 81 bytes smaller than `parameter_intro`, but has 16 more tokens and 47 more AST nodes. Inferred Export itself is nine bytes larger than the matched explicit-Name version; its advantage is the removal of an equality-proof input. These are distinct tradeoffs. Gzip sizes and exact source hashes appear in `evidence/metrics.json`; they are not used to claim mathematical minimality.

The matched liberal control differs only in retaining the supplied introduction witness and its validation. This comparison measures the new witness-removal theorem. Comparisons with `paramete r_intro` also change the discharge rule and certificate organization; they are not isolated experiments. The corpus contains a 508-byte compact scalar control with a different public-validation contract. The new presentation does not beat that byte record. The complete normalization and validation code is included in every stated measurement; the Python runtime is the same shared background as in the repository's metric.

The optional fuel checker decrements a nonnegative budget at every recursive normalization and proof-checking entry. The bounded relation is primitive recursive on finite certificates. Every successful unbounded derivation has some finite sufficient budget, and a successful bounded derivation is also unbounded-valid. These assertions concern the idealized algorithms with unbounded finite storage; Python's stack and memory limits can reject a sufficiently large valid certificate.

11 Executed evidence and claim boundaries

The delivered portable reproduction passed the following checks:

- 24 accepted and 16 rejected certificate fixtures; 80 standalone checker runs across ordinary and optimized Python; three open-scope checks.
- Six source-to-target proof translations, including a nontrivial injective scope embedding, and a translator CLI check.
- All 24 positive certificates replayed against each of the two matched controls; 48 exact-budget/one-short checks and nine malformed-fuel rejections for the bounded checker.
- An independent named-variable implementation agreed on 400 normal forms, 1200 numerical evaluations, 480 binder openings, 320 successor contractions, and two capture regressions.
- A separate 25-case syntax-rejection suite. Some cases overlap the main sixteen; these counts are not added as distinct attacks.
- All 24 main positive certificates translated to backward Name, two additional erased-template acceptances, ten negative cases, and 72 ordinary/optimized CLI runs. A bounded template-completion oracle examined 5000 templates, found 149 valid endpoint pairs (including 64 replacements containing the fresh parameter), and confirmed the exact Name endpoint identity in all 149 cases. Ten valid pairs erased an auxiliary raw parameter.

The examples include arbitrary introduction at 1 and 2, genuinely used progress hypotheses, capture avoidance under D, and $\forall a\, \forall b\, (a = b \to S(b) = S(a))$ obtained through a nontrivial inferred export. Negative tests exercise a fresh variable escaping the default export, a false naming pattern, the exact-$0 = 0$ mode guard, malformed erased syntax, and Boolean/float aliases.

The source-to-target translator is a recursive algorithm on arbitrary validated finite source proof trees. The following executions illustrate its actual expansion; the byte counts concern minified proof JSON alone.

Example	Old nodes	New nodes	Old bytes	New bytes
Introduction at witness 1	2	15	74	529
Conditional copy, depth 1	7	46	368	2527
Used progress hypothesis	15	80	421	2868
Universal successor congruence	25	142	1335	8909
Nontrivial open-scope embedding	4	17	76	501

Thus a smaller trusted interface can require longer certificates. `code/translate_witness.py` implements both the substantive translation and the easy reverse inclusion that inserts the already selected witness. The proof producers are untrusted: every generated certificate is replayed by the complete checker. All outputs and counts are recorded in the JSON files under `evidence/`.

These checks support the implementation and make the unusual proof construction inspectable. Finite tests do not establish the general metatheorems. Conversely, the written proofs do not certify every byte of the Python implementation. The delivered material deliberately separates the new proof, the executable certificate evidence, and the upstream Lean results. No new Lean or Rocq verification is claimed.

12 Further research questions

1. **Formalize conditional copy independently of arithmetic.** The proof only needs an exact successor predecessor, numerical gates, clause specialization, newest-parameter Close, and ordinary elimination. A small generic Lean interface for these operations could make the witness-removal theorem reusable across several native presentations.
2. **Reduce the remaining specialization interface.** Use still supplies an arbitrary term. The present proof specifically specializes its outer closure at $S(t)$. Replacing that operation by a fixed specialization requires a new way to deliver t; deleting it is not justified by the theorem proved here.
3. **Quantify proof expansion.** The translation regenerates a smaller branch inside extra scopes and can duplicate it. The finite certificates demonstrate feasibility, but no polynomial bound on general certificate size is asserted. Sharing source proof builders through a checked DAG may improve size without enlarging the trusted mathematics.
4. **Normalize less.** The literal equations $P(St) \equiv t$ and the selector endpoints do the work. A canonical syntax that guarantees these particular reductions while avoiding full normalization could reduce the trusted checker. It would need a new coherence proof and a fresh size comparison.
5. **Separate interface minima from program minima.** Three rule families with fewer supplied fields do not imply a coordinatewise minimum in bytes, tokens, or AST nodes. A useful next benchmark would fix the syntax-validation contract and measure several straightforward implementations of all competing calculi.

A Complete selected checker

This listing is the complete measured source, including its input and acceptance entry point. The readable source, matched controls, bounded variant, proof constructors, and certificate fixtures are in the archive.

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or size in ((1, 2) if form else (3,)))
    a = [walk(b, f, d + 2 * (size != 2 and j == size - 1), size == 1) for j, b in enumerate(a)]
    if size == 3:
        c, b, h = a
        if c == 0:
            return [0, b, 0]
        if type(c) is list and c[0] == 0:
            return sub(sub(h, c[1], 1), walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, n, form=False):
```

```
    need(walk(a, lambda i, d: need(1 - n <= i <= d) or i, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: walk(t, lambda j, e: j + d if j > e else j) if i == d + 1 else i - (i
    > d + 1), depth)

def prove(p, g):
    n = len(g)
    b = -n
    code, *args = p
    if type(code) is int:
        t, q = args
        need(0 <= code < n)
        syntax(t, n)
        base, B = g[code]
        if base:
            need(prove(q, g) == base)
        A = sub(B, t)
    elif code:
        r, = args
        B = sub(code, 1 - n, 1)
        base = sub(B, 0)
        hyp = sub(B, b) if base == [0, 0] else base
        need(prove(r, g + [([], hyp)]) == sub(B, [0, b, 0]))
        A = [code]
    else:
        need(code == [])
        q, r = args
        D, = prove(q, g)
        B = sub(D, b, 1)
        base = sub(B, 0)
        row = sub(B, [0, 0, 0])
        A = walk(prove(r, g + [(base, B)]), lambda i, d: row[1] if base == [0, 0] and row[0] == i == b
     else i)
    syntax(A, n, True)
    return A

def check(data):
    goal, proof = data
    need(repr(prove(proof, [([], [1, 1])])) == repr(goal))
if __name__ == '__main__':
    check(json.load(open(0)))
    print('ACCEPT')
```

B Complete backward Name checker

This is the complete 1907-byte normalized source of K_N, including its JSON input and acceptance entry point. The standalone file is `code/checker_backward_name.py`. Section 9 proves that its template instruction represents Name exactly.

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
```

```
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or size in ((1, 2) if form else (3,)))
    a = [walk(b, f, d + 2 * (size != 2 and j == size - 1), size == 1) for j, b in enumerate(a)]
    if size == 3:
        c, b, h = a
        if c == 0:
            return [0, b, 0]
        if type(c) is list and c[0] == 0:
            return sub(sub(h, c[1], 1), walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, fresh, form=False):
    need(walk(a, lambda i, d: need(fresh < i <= d) or i, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: walk(t, lambda j, e: j + d if j > e else j) if i == d + 1 else i - (i
    > d + 1), depth)

def prove(a, p, g):
    fresh = -len(g)
    syntax(a, fresh, True)
    match p:
        case ['u', j, t, premise]:
            need(type(j) is int and 0 <= j < len(g))
            syntax(t, fresh)
            base, body = g[j]
            need(a == sub(body, t))
            if base:
                prove(base, premise, g)
        case ['e', body, template, t, major, eq, branch]:
            prove([body], major, g)
            body = sub(body, fresh, 1)
            local = g + [(sub(body, 0), body)]
            prove([fresh, t], eq, local)
            prove(sub(template, fresh), branch, local)
            need(sub(template, t) == a)
        case ['i', branch]:
            need(len(a) == 1)
            body = sub(a[0], fresh + 1, 1)
            base = sub(body, 0)
            hyp = sub(body, fresh) if base == [0, 0] else base
            prove(sub(body, [0, fresh, 0]), branch, g + [([], hyp)])
        case _:
            need(False)
goal, proof = json.load(open(0))
prove(goal, proof, [([], [1, 1])])
print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, pinned repository README, commit `8fa5ade009b5cceba7b68f07b6a834fe77cf10c1`. https://github.com/VladimirRes

hetnikov/ProveIt/blob/8fa5ade009b5cceba7b68f07b6a834fe77cf10c1/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[2] *parameter_intro*, especially `core.tex`, `normalization.tex`, and `comparison.tex`, in the pinned ConciseIPRA Research directory. https://github.com/VladimirReshetnikov/ProveIt/tree/8fa5ade009b5cceba7b68f07b6a834fe77cf10c1/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/parameter_intro.

[3] *first_instance*, *clause_discharge*, and *witness_discharge*, pinned ConciseIPRA Research packages. The deterministic export comparison is formalized as `NativeExportClauses.lean`, theorem `xcprf_iff_nprf`. https://github.com/VladimirReshetnikov/ProveIt/blob/8fa5ade009b5cceba7b68f07b6a834fe77cf10c1/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeExportClauses.lean.

[4] *uniform_recursion*, especially `report.tex`, the complete counter-recursor checker, and the 508-byte compact control, pinned ConciseIPRA Research directory. https://github.com/VladimirReshetnikov/ProveIt/tree/8fa5ade009b5cceba7b68f07b6a834fe77cf10c1/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/uniform_recursion.

[5] *rootfree_clausal* and *reflexive_base*, pinned ConciseIPRA Research packages: introduction selected by a normalized zero row. The exact-$0 = 0$ test used here is the narrower version of that existing mechanism.

[6] ConciseIPRA Lean development, especially `NativeDiagonal.lean`, `NativeFullNormal.lean`, `NativeExportClauses.lean`, `NativeBiInterpretation.lean`, and `BiInterpretation.lean`, at the pinned commit. https://github.com/VladimirReshetnikov/ProveIt/tree/8fa5ade009b5cceba7b68f07b6a834fe77cf10c1/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA.